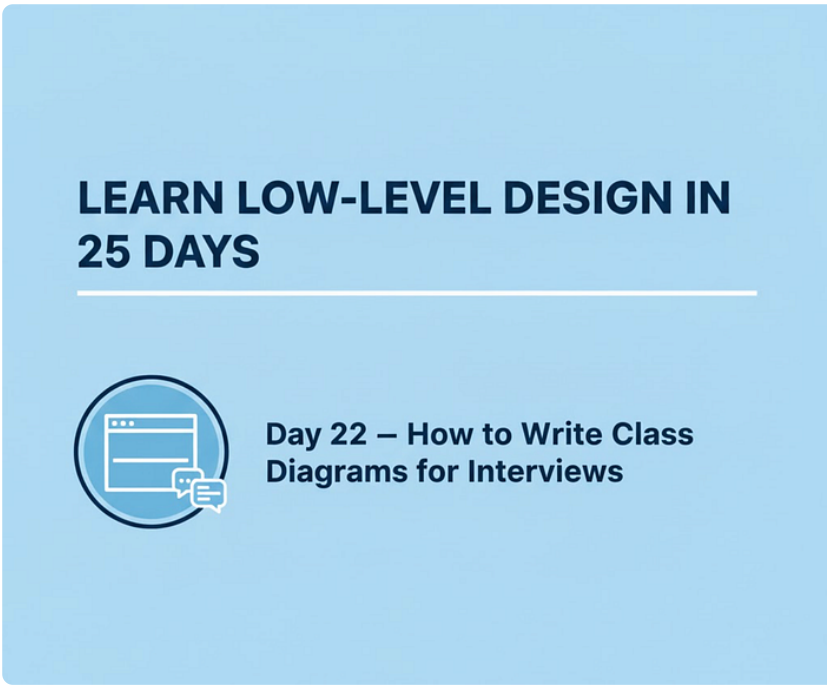



# Learn Low-Level Design in 25 Days: Day 22—How to Write Class Diagrams for Interviews

(Part of the 25-day LLD Bootcamp: Save the full series here)

**TechPreneur**

Published November 4, 2025

Free: No

Read on Freedium · original

# Contents


1. The Interview Moment Every Developer Dreads 0

2. Core Concepts — Why Class Diagrams Matter 0

3. The Ingredients of a Class Diagram 0

Quick Symbol Guide 0

4. Example: Designing a Chat System (Simplified) 0

Step 1: Identify Key Entities 0

Step 2: Define Relationships 0

Step 3: Draw the Diagram 0

Step 4: Key Takeaways from the Diagram 0

5. How to Draw Class Diagrams in Interviews 0

6. Try It Yourself 0

7. Summary + What's Next 0

Key Takeaways 0

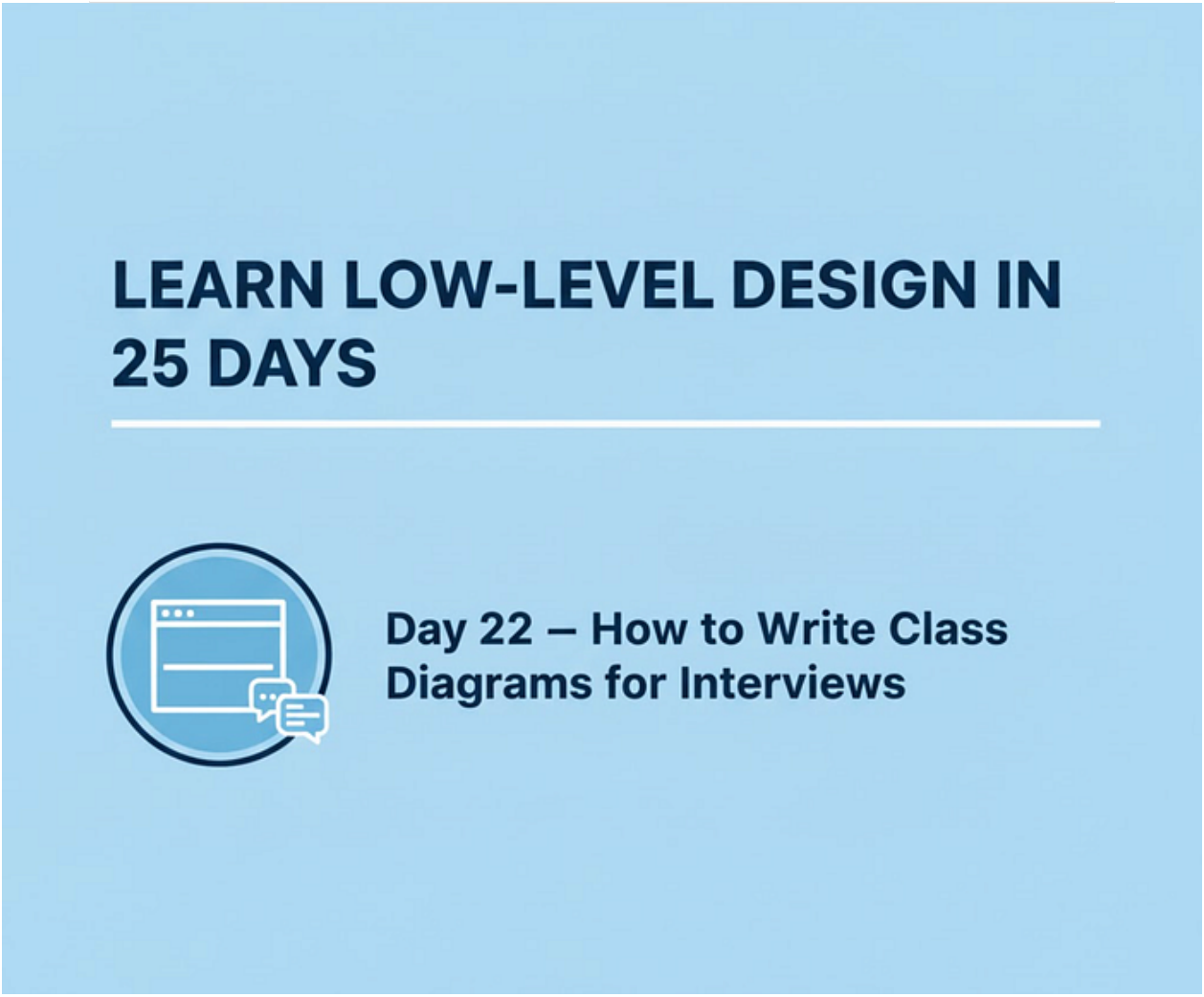
## 1. The Interview Moment Every Developer Dreads

Picture this: you're in a system design interview, breezing through the functional requirements. You've just described how messages flow in a chat app or how notifications trigger in an e-commerce platform. Then the interviewer says — "Cool. Can you draw the class diagram for that?"

Suddenly, your confidence dips. Should `User` have a list of `Messages`? Should `ChatRoom` own `Users` or just reference them? What goes into `MessageService` versus `Message` itself?

This is the moment where great developers separate themselves — not by remembering syntax, but by **communicating structure clearly**. And that's what class diagrams do best: they show how your objects interact, who depends on whom, and how data and behavior flow in your design.

Welcome to **Day 22 of Learn Low-Level Design in 25 Days** — today, we'll master how to write and read **class diagrams for interviews**. By the end, you'll be able to confidently sketch clean, readable diagrams that make your design ideas instantly click with your interviewer.




## 2. Core Concepts — Why Class Diagrams Matter

At its core, a **class diagram** is a blueprint of your design. It captures **entities (classes)**, their **attributes**, **methods**, and **relationships** (like association, inheritance, or composition).

Think of it like a city map:

- **Classes** are buildings.
- **Attributes** are what's inside.
- **Methods** are what each building does.
- **Arrows and lines** show how they're connected.

When you design a feature — say, a ride-booking flow or an order management system — a good class diagram helps:

- You think **in terms of objects**, not APIs or tables.
- Interviewers see **clarity and ownership** — who's responsible for what.
- Everyone aligns quickly on **responsibilities and interactions**.

## 3. The Ingredients of a Class Diagram

Let's break down the anatomy of a class diagram:

```
+----------------+
|   ClassName    |
+----------------+
| - attribute1   |
| - attribute2   |
+----------------+
| + method1()    |
| + method2()    |
+----------------+
```

- "-" means private
- "+" means public
- "#" means protected

Relationships are drawn like this:

```
User 1 ------ * Message     (One-to-Many Association)
Message <|---- TextMessage  (Inheritance)
ChatRoom *----* User         (Many-to-Many Association)
```

### Quick Symbol Guide

RelationshipSymbolExampleAssociation — `User – Order`One-to-Many1 — *`User 1 – * Message`Aggregation (has-a)◇ `Team ◇– Player`Composition (owns)◆ `Library ◆– Book`Inheritance<—

## 4. Example: Designing a Chat System (Simplified)

Let's apply it to something familiar — a **Chat System** (like WhatsApp or Slack).

### Step 1: Identify Key Entities

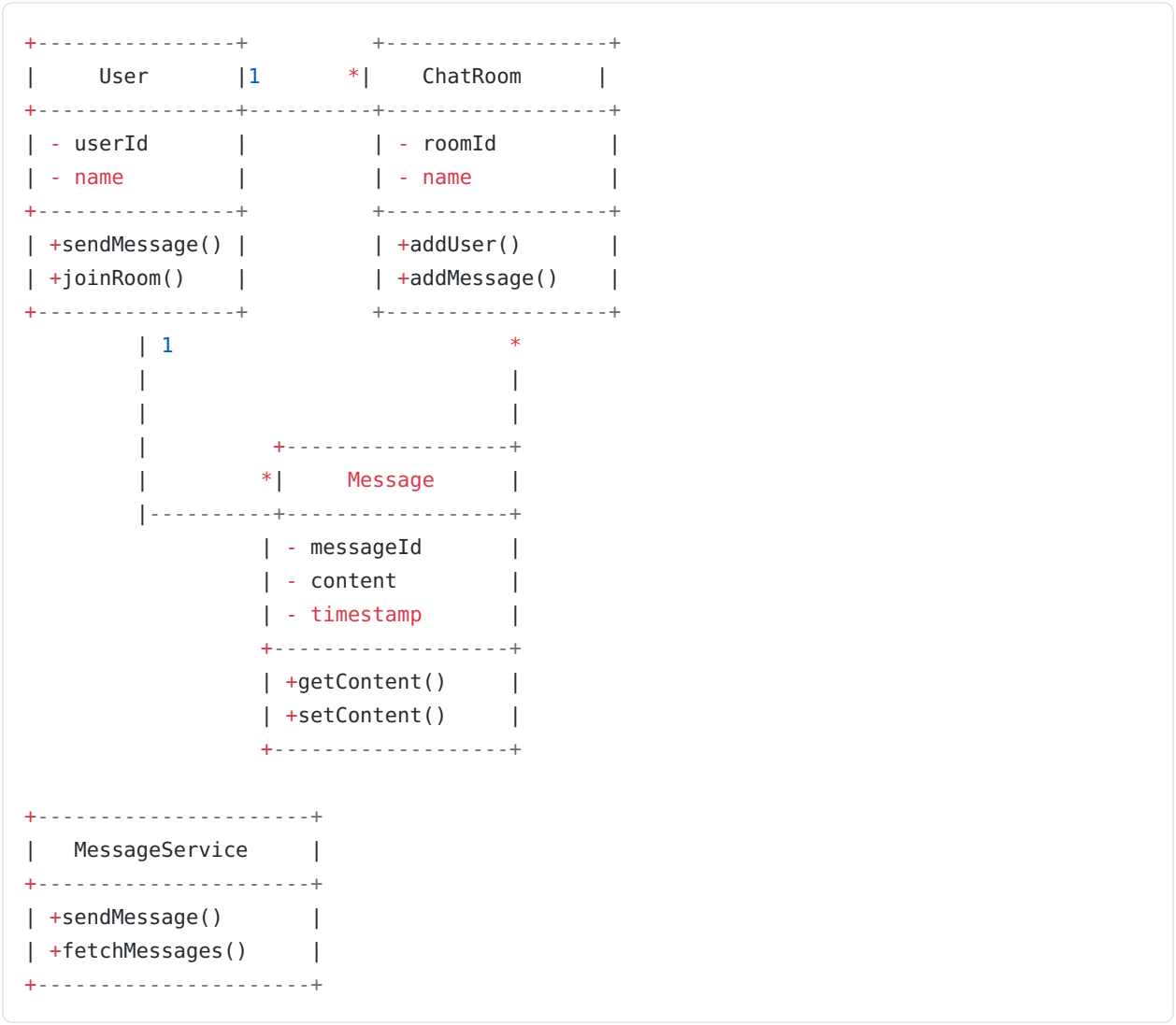
- **User**
- **ChatRoom**
- **Message**
- **MessageService**

### Step 2: Define Relationships

- A `User` can belong to many `ChatRooms`.
- A `ChatRoom` has many `Users` and many `Messages`.

- `MessageService` handles sending and retrieving messages.

### Step 3: Draw the Diagram




### Step 4: Key Takeaways from the Diagram

- **Encapsulation**: Each class owns its own data and logic.
- **Association clarity**: `User` ↔ `ChatRoom` is many-to-many.
- **Separation of concerns**: Business logic (in `MessageService`) is separate from data holders (`Message`, `User`).

## 5. How to Draw Class Diagrams in Interviews

Here's a simple, repeatable framework:

1. **Start with nouns** → these are likely your classes. (e.g., User, Message, Order, Product)
2. **Add verbs** → these become methods. (e.g., sendMessage(), placeOrder())

3. **Identify relationships** → who "has" whom? who "uses" whom?
4. **Show multiplicities** → 1:1, 1:N, N:M.
5. **Refine access modifiers** → - for private, + for public.
6. **Keep it clean** — no clutter. Only show what adds value.

You don't need a perfect UML tool. A neat whiteboard or a simple ASCII diagram (like above) is often enough in an interview.

## 6. Try It Yourself

**Challenge 1:** Design a class diagram for a Food Delivery App (like Swiggy). Hint: You'll likely need classes such as `User`, `Restaurant`, `Order`, `DeliveryPartner`, and `Payment`.

**Challenge 2:** Take one of your past projects. Sketch 3–5 classes and their relationships. Ask yourself:

- Which classes own others (composition)?
- Which are shared (association)?
- Which just extend functionality (inheritance)?

## 7. Summary + What's Next

### Key Takeaways

- Class diagrams are **the language of design** — they communicate structure better than words.
- Focus on **clarity**, not completeness.
- Use multiplicities and access modifiers to show intent.
- Always tie the diagram back to real-world behavior — how these classes collaborate at runtime.

Use class diagrams to **think before you code**, and you'll notice how your LLD answers start sounding like architecture discussions instead of syntax explanations.

**Tomorrow (Day 23):** We'll dive into **Trade-offs: Performance vs Readability in LLD**, and explore how real systems decide between speed, clarity, and maintainability.

You can follow or bookmark the full series here: Learn Low-Level Design in 25 Days (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)